

WeConnect 2.0 — Quality Assurance

Phases 1 and 2 • 25 September 2026 • For client review

Status

Phase 1 — read-only	Complete
Phase 2 — transactional	Complete
Environment	Isolated, version-matched
Client data written	None

Coverage

Static routes swept	235 / 235
Dynamic routes	17 / 17
API endpoints probed	71
Self-verification controls	46
Business objects driven	12

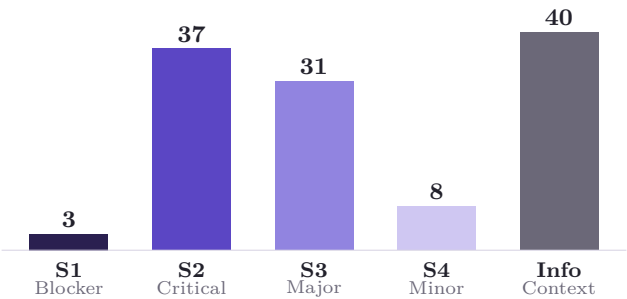
In brief

What we found. 119 findings. Three are confirmed breaches of the centre permission boundary — an account acting on another centre’s records, checked against the database. They share one cause: the application code goes around a permission mechanism that works.

Fix these four first. They are minutes of work each. WC-088 a one-word typo that breaks Enquiry Form status changes at two sites • WC-068 remove `allow_guest` from three endpoints publishing your full API specification • WC-023 stop returning stack traces to callers • WC-121 replace the CORS wildcard with your two frontend origins.

What we need from you. Sign the characterisation document — Part 1 is one page, and asks you to confirm seven answers we worked out ourselves and answer **two** questions we could not. Everything else waits on that.

Findings — 119 total



S1 Blocker — unauthorised action or data loss. S2 Critical — security control missing or key logic failing. S3 Major — significant defect with a workaround. S4 Minor — presentation or hygiene. Info characterisation, no action.

Critical (S1) — three confirmed breaches of the centre permission boundary

We performed each of these on an isolated copy of the platform, signed in as an ordinary staff account, against a record belonging to a centre that the account has no permission for. In each case we then checked the database directly to confirm that the change had actually been made, rather than relying on what the API reported. None of this was done against your data.

S1-1 WC-095 An account raised another centre’s invoice into the financial approval pipeline

The `sp` account is permitted for **QA East** and **QA West** only. It called `raise_invoice` on an invoice owned by **QA North**. The platform accepted the call and advanced the invoice through its approval workflow.

Endpoint	<code>youth_skilling.api.invoice.raise_invoice</code> — PUT
Replied	<code>success_key:1</code> — "Skilling Invoice SK-INVOICE-00003 raised successfully."
Database after	<code>current_status = Raised, approval_status = Verification Pending</code>
Reproduce	<code>reproductions/s1-invoice-raise.mjs</code>

Why it matters. The invoice has entered the financial approval queue, and the account that put it there

has no authority over the centre that owns it. The endpoint takes the centre from the request and checks only that the invoice exists under that centre. It does not check whether the caller is permitted for that centre.

Fix. Take the centre from the caller's own permissions rather than from the request, and remove the `ignore_permissions` flag from the save so that the framework's own check applies.

S1-2 WC-084 An account closed another centre's batch

The same account, with the same permissions, called `close_batch` on a batch owned by **QA North** and closed it.

Endpoint	<code>youth_skilling.api.batch_management.close_batch</code> — POST
Replied	<code>success_key:1</code> — "Batch closed successfully."
Database after	<code>batch_status = Closed</code>
Reproduce	<code>reproductions/s1-batch-close.mjs</code>

Why it matters. Closing a batch completes it, which affects the status of the young people enrolled in it and the invoicing that follows. The endpoint already reads the batch's owning centre in order to build its reply, but it does not compare that centre against the centres the caller is permitted for.

Fix. Compare the batch's centre against the caller's permitted centres before closing it, and remove the `ignore_permissions` flag from the save.

S1-3 WC-048 An account read another centre's youth counselling records

An Outreach Coordinator account, permitted for two centres, requested the counselling list and received **ten of ten records from centres it has no permission for**. Each record named a youth and included the type and outcome of their counselling session, along with free-text remarks about them.

Endpoint	<code>facilitator_and_counsellor.api.counselling.list.get_counselling_list</code>
Replied	HTTP 200 — 10 of 10 records outside the caller's permitted centres
Confirmed	Reproduced three times; cause verified against the live platform, read-only
Fields returned	<code>youth_name</code> , <code>counselling_type</code> , <code>remarks</code> , <code>last_call_remarks</code> , <code>status</code> , and eight more

Why it matters. These records concern youth and include counselling notes about them. They were released across a permission boundary that the platform itself publishes to the account at sign-in. We would treat this as a data-protection issue as well as a software defect, and it may carry reporting obligations that are for the foundation to assess.

The cause here is different from the other two, and so is the remedy. The handler already reads the caller's permitted centres and filters correctly on them. That filter is never reached, because of the line immediately above it, which returns early for any caller holding the **System Manager** role. Staff accounts hold that role because the permission model gives them no working alternative: of the 293 doctypes the platform installs, 178 grant permissions to **System Manager** and no other role, and the remaining 115 grant none at all.

Fix. Remove the role check from this and similar handlers. That change has to be made together with proper per-role permissions on the doctypes, because on its own it would leave staff accounts unable to see anything at all. The next section sets out the wider position.

We recorded only the names of the fields returned and the number of records. No values from any youth's record were copied into this report or into any file we hold.

All three share the same underlying cause. Two of them are write operations that switch the permission check off, and the third is a read operation that skips it for a role that every account holds. Changing how authorisation is enforced would address all three together, rather than requiring three separate fixes.

The underlying cause

The framework’s own permission mechanism is working correctly. When a write goes through the ordinary save path, an attempt to change another centre’s record is refused, and the error names the centre that blocked it. This holds even for accounts that carry the **System Manager** role. In every case we found, the problem is that the application code goes around that mechanism rather than through it. It does so in two ways, and a third gap sits underneath both.

On writes

Thirty endpoints that change data pass the `ignore_permissions` flag to the call that writes, which switches the framework’s check off, and they perform no check of their own beforehand.

- We called 36 of them as an ordinary user. None refused the call.
- Driving the state-changing ones against another centre’s records, three made the change and none refused; two could not be judged because the test data did not support them. Counting the two critical findings above, five endpoints are confirmed to change another centre’s data.
- Fifteen of the 36 also return a server error, with a stack trace, when called with no input at all.

On reads

Filtering by centre is skipped for any caller holding the **System Manager** role. We counted 45 places where this happens, across 26 files in seven of the eight applications. The filters themselves are present and correct; they simply sit inside a condition that excludes that role. This is what caused the counselling finding above.

On the records themselves

The doctypes define no rules of their own, so nothing catches what the endpoints let through. Every one of 174 attempted status changes was accepted across six doctypes, including **Paid** moving back to **Not Raised Yet** on an invoice. Ten of twelve self-contradictory values were stored without complaint, among them a negative age, a birth date in the future and a negative course cost. Constraints written here would apply to every write path at once.

What we would recommend, and in what order. Authorisation should be enforced in one place, through a decorator or a shared handler that every data-changing endpoint passes through, rather than being written again in each endpoint. The role check should then be removed from the read paths. Both of those changes depend on first giving each role real permissions on the doctypes, because until that exists, removing the role check would leave staff unable to use the platform at all. The validation and transition rules are independent of that sequence and can be added at any point; they are also the cheapest of the three, because they are written once per doctype rather than once per endpoint.

S2 findings by area



The highlighted group is the one we would address first.

Why this affects every account

Of the **293** doctypes the platform installs, **178** grant permissions to **System Manager** and to no other role, and the remaining **115** grant none at all. A staff account therefore cannot do its job without holding the role that turns the filtering off.

Critical findings (S2) — all 37, with remediation

Finding	Fix
Authorisation and data exposure — 10	

Finding	Fix
Mutating endpoints bypass permissions with no authorisation check (30 endpoints) Data scoping is disabled for System Manager at 45 sites in 7 of 8 apps — and 178 of 293 doctypes grant that role alone, so every staff account holds it Any authenticated account, including an external partner, can trigger a bulk export of programme data to a Google Sheet Financial endpoints share the confirmed cross-centre write gap (raise_invoice, prepare_tax_invoice) Personas receive records belonging entirely to centres they are not permitted for Any authenticated user can delete any file by URL — no ownership check An unauthenticated endpoint attempts to provision a System Manager account The complete API specification is published to unauthenticated callers The permission design forces over-privileged accounts or bypasses permissions entirely add_interest writes every field the caller sends onto the Youth record, then saves with ignore_permissions=True; confirmed setting a field the endpoint never declares Reliability and correctness — 13 Fields accept values that contradict themselves: negative ages, birth dates in the future, a capacity floor above its ceiling, negative course costs and instalment percentages of 150% and -20% No state-transition control exists: 174 of 174 attempted status changes were accepted across six doctypes, including Paid→Not Raised Yet and Cancelled→Paid on invoices Enquiry Form status change is completely broken by a variable typo Several endpoints return 403 over HTTP for every caller, Administrator included, though the function works when invoked directly Backend returns 500s under modest concurrency; sensitivity quantified by dose-response SYC /curriculum data API returns 500 14 routes render blank on direct load; 2 group-based routes fail identically in both apps Opening the invoice screen attempts to create a tax invoice with no user action Target Data API returns 404; widget shows “Data not available” Endpoints crash rather than validate on missing input (15 of 36) A field returns the sentinel string “NA” where a list is expected State transitions are not guarded on current state No rate limiting on authenticated requests, on a backend that fails above three concurrent sessions Configuration and handover — 7 The permission model exists in no repository Runtime configuration required to operate the platform is not in the repositories Application code gates on roles that do not exist on a fresh install An installed application is absent from the repositories handed over Attendance cannot be recorded on a fresh install — the code writes Youth.status values that ship in no master	Central authorisation: one decorator or base handler every mutating endpoint passes through. Remove the role check from scoping paths, <i>together with</i> a real per-persona permission model. Removing it alone locks staff out. Require an explicit administrative role for sync_to_lcf_mastersheet, and log every invocation with the calling account. Derive center from the caller’s permitted scope, not from the request. Two of the three handlers are the System Manager bypass above, not a missing filter. The third scoped correctly under test; its cause is unresolved. Verify the caller has permission for the file’s parent document before deleting. Require an authenticated, authorised caller; never grant System Manager from it. Runtime blocks it today; the code path remains. Remove allow_guest=True from the three swagger_spec functions. Define per-persona roles with real doctype permissions, and stop relying on bypass. Reject keys outside the declared set before writing — update_issue in the same codebase already does this — and drop ignore_permissions. Put the constraints on the doctype in a validate method, so every write path inherits them rather than each endpoint repeating them. Define permitted transitions once on the doctype. Frappe’s Workflow facility also supplies the transition history the invoice states need. ids = json.loads(ids) — a one-word correction, plus a test that calls the endpoint. Root-cause the API-layer rejection; these endpoints are unreachable over HTTP today. Size the worker pool for expected concurrency; return 503, not 500, under load. Fix the server error in the curriculum data handler. Root cause is null navigation state — read identifiers from URL parameters, not router state, and guard the read. Load the screen with a GET; perform invoice preparation only on explicit user action. Restore the endpoint or remove the widget. Validate presence explicitly and return 400 naming the missing field. Return []. A field’s type should not change with its emptiness. Refuse to raise an already-raised invoice; guard each transition on the state it starts from. Apply per-account and per-IP limits sized to the concurrency the backend can sustain, and paginate the unbounded youth endpoint. Export roles, role profiles and permission rules as app fixtures. Ship it as Frappe fixtures — the mechanism exists for exactly this. Ship the roles and the permission rules that reference them. Declare required_apps in hooks.py and record the deployable set. Ship the Youth Status master as fixtures, including every value the code writes. Until then every attendance save is rejected.

Finding	Fix
A ninth role profile (Placement Youth Coordinator) is routed to <code>/pyc/</code> , an application that does not exist Approval flows name at least 15 Approval Category records as string literals, and none ships — so every approval-driven flow fails on a fresh install	Decide whether the persona is live. If it is, the application is missing; if not, remove it from the routing table and backend helpers. Ship the categories as fixtures with the correct <code>ref_doc</code> , replace the literals with constants, and reconcile Move to Another Batch / Move to another Batch .
Environment and data protection — 3 The environment → API mapping is shifted by one in all 9 repositories: <code>.env.uat</code> calls dev, <code>.env.prod</code> calls uat, and no config names a production API Real programme data, including minors' records, is present in UAT A long-lived API secret is stored in <code>localStorage</code>	Correct each environment's mapping and add the missing production configuration. Confirm which config each deployed frontend is built from. Anonymise UAT, or confirm it is a production copy and restrict access accordingly. Move to a short-lived, <code>httpOnly</code> session cookie.
Frontend and accessibility — 4 Server stack traces returned to callers in the standard error envelope The client-side auth guard is hardcoded to <code>true</code> — disabled in two persona apps Systematic WCAG critical violations, concentrated in shared navigation The frontend is served with no security headers: no CSP, HSTS, <code>X-Frame-Options</code> or <code>X-Content-Type-Options</code>	Disable client-facing traceback rendering in the deployed site config. Restore the guard and treat 403 like 401 in <code>axios-setup.js</code> . The backend does enforce auth, so this ships a broken shell, not an exposure. Correct the sidebar list markup, add <code>alt</code> to nav icons. One shared component fixes most instances. Add them at the nginx layer. CSP matters most here, because it is the control that limits what injected script can do with the token held in <code>localStorage</code> .

Where to start

Every finding in the register carries an identifier and a priority. Priority is not severity: it weighs how much the fix costs against what leaving it costs. Seventeen items are P1. Four of them are a few minutes' work each and are worth doing this week, whatever else is decided.

Quick wins — small changes, disproportionate effect

ID	What to do
WC-088	Correct <code>id = json.loads(id)</code> to <code>ids = json.loads(ids)</code> . It appears at two sites, not one; fixing either alone leaves the other path broken. Restores a feature that fails on every call.
WC-068	Remove <code>allow_guest=True</code> from the three specification endpoints. Takes the full API surface out of public view.
WC-023	Turn off client-facing traceback rendering. Stops every error handing back internal paths and query structure.
WC-121	Replace the CORS wildcard with an explicit list of your two frontend origins. One configuration line.

The rest of P1

Confirmed breaches: WC-048, WC-084, WC-095 — the three in the section above. **Reachable without an account:** WC-077 provisioning, WC-078 file deletion by URL. **Data leaving its boundary:** WC-035 cross-centre records, WC-115 bulk export, WC-003 real PII in UAT. **Writes nobody asked for:** WC-038 invoice screen. **Unusable today:** WC-043 blank routes. **The programme of work:** WC-073, WC-079, WC-085, WC-105 — these are the authorisation rebuild, and the three confirmed breaches cannot close without them.

The repository includes `verify/`, which re-checks the mechanically checkable findings against your own environment and reports which are fixed. It needs only Node, issues read-only requests by default, and is safe to point at any environment. A check that cannot run reports an error rather than a pass.

What we need from you

- Fix the three critical findings and the two bypasses behind them.** The reproduction scripts included with this report will confirm when each one is closed.
- Give each role proper permissions on the doctypes, then remove the System Manager check from the filtering code.** These two need to be done together. Because 178 of the 293 doctypes grant permissions to that role and no other, removing the check on its own would leave staff unable to use the platform.

3. **Put the access-control configuration into the repositories.** Roles, role profiles, permission rules, naming series and the master data the code depends on should ship as fixtures. Four of the applications already use that mechanism for other settings, so this is a matter of extending what is there rather than introducing anything new.
4. **Correct the environment configuration.** Each environment's frontend should point at its own API, and a production configuration needs to be added. We would also like to know which configuration each deployed frontend is currently built from, and whether the UAT database holds real programme data.
5. **Review and sign the characterisation document.** It marks 15 decisions. We went back through them and answered what we could from the code and the data: seven are settled and need only your confirmation, six are narrowed to a smaller question, and **two genuinely need you** — whether there is an escrow or handover arrangement for `fintoo_analytics_bridge`, and whether each role has the right screens. Section 12 of that document sets out which is which, and what we found when we stopped asking and went looking.
6. **Tell us how deletion is meant to work**, since no object can be deleted through the API at present, and whether `Placement Youth Coordinator` is a role you are still using.

Phase 3 — once the above is in hand

The work below is what Phase 3 covers. It can begin once the characterisation document is signed and the three critical findings are being addressed.

- **Cross-browser and device matrix.** Playwright covers Chromium, Firefox and WebKit at no cost; BrowserStack is needed only for real iOS and Android hardware on a curated subset.
- **Continuous integration.** The suite on a defined trigger, with results visible to your team.
- **Regression closure.** Re-run the reproduction scripts against the remediated build and confirm each S1 closed.
- **Handover.** Documented suite, runbook and final closure report.